

Clemson University

CPSC 4200 - Computer Security Principles Homework 2 Assignment: Cryptography

Spring 25'

Trent Brown | stb2@g.clemson.edu
Ben Simmons | busimmo@g.clemson.edu

Contents

1.	Padding Oracle Attack	1
2.	Bleichenbacher's Attack	2

1 **Padding Oracle Attack**

This report details the exploitation of a Padding Oracle vulnerability in a web API. The attack leverages a weakness in the site's encryption scheme, allowing an attacker to decrypt messages without knowing the key. We break down our methodology, the steps we took to perform the attack, and the division of work between the group.

The target website encrypts messages using AES-128 in CBC mode with PKCS #7 padding. If an invalid padding error is exposed during decryption, an attacker can use this information to recover the plaintext byte-by-byte. The website provides an endpoint:

`http://cpssc4200.mpese.com/username/paddingoracle/verify?message=message`

where the message is an encrypted payload. By modifying the ciphertext and observing the server's response, we can infer the correct padding and extract the original plaintext.

Trent focused on understanding the cryptographic principles behind padding oracle attacks and implementing the script to automate the attack. Ben analyzed the HTTP responses from the server and tested different ciphertext manipulations to determine how padding errors were returned. We began by sending a valid encrypted message to the API. Then, we manipulated the last byte of the final block and resent it. If the server returned an "invalid_mac" error, we confirmed the presence of a padding oracle vulnerability.

Our methodology consisted of a script to iteratively modify the ciphertext and send requests to the API. The script:

- Iterated through possible byte values (0x00 - 0xFF) in the last byte.

- Checked the server's response for a padding error.

- Used the discovered padding to infer the actual plaintext values.

- Repeated the process for each byte in the last block before moving to the previous block.

- Once the last block was decrypted, we used the known plaintext to XOR against the previous ciphertext block, ultimately revealing the entire message.

Some challenges that we encountered were the server response times, we noticed that each period took a long time to run the program through the server, which caused delays.

We also ran into issues with Error Handling. Some responses were inconsistent, requiring additional validation logic in our script when testing.

Lastly, Padding. We realized that the logic to remove the PKCS #7 padding caused issues in our results.

```

benjaminusimmons@Benjamins-MBP-2 project2_starter-1 % python3 padding_oracle.py "http://cpsc4200.mpesa.com/busimmo/paddingoracle/verify" "c0a3606bb1ca70dc53647632db45cccc3aa61d9ae5d78922be5d8868dc58a9a1e52ba032359903862cbd1144e4245d0b9c085449c8e42ec85d5fc84f6f1ec13d1154a31efead864bed12e7e8dbd5f15b745fc45e9621f28978501f80ed8ac38e47ef6bc94fd00d34defd"
/Users/benjaminusimmons/Library/Python/3.9/lib/python/site-packages/urllib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only supports OpenSSL 1+, currently the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
warnings.warn(
(: She turned me into a newt! K: A newt? T: Well, I got better.Ig,6cc
benjaminusimmons@Benjamins-MBP-2 project2_starter-1 %

benjaminusimmons@Benjamins-MBP-2 project2_starter-1 % python3 padding_oracle.py "http://cpsc4200.mpesa.com/busimmo/paddingoracle/verify" "0e057bb7cb0072ee76cf688a61543b130c3ee33c7ac40720da84eda61b1a9588ad2fd586d1a56aee2c62ca95c27fdb6dc1c0a6be9d0ed8e9f6e3bd63df23629ed164a19ae3dd18ee4a0d9303ad332b4506624fe182e61d05968840e139747b6d64d136fb61445f3174321f84b487106739d47c52b87401b5f4ca764f6cb933fb52001a8423a615b75fb"
/Users/benjaminusimmons/Library/Python/3.9/lib/python/site-packages/urllib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only supports OpenSSL 1+, currently the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
warnings.warn(
We elves try to stick to the four main food groups: candy, candy canes, candy corns and syrup.))j,+Y
io
benjaminusimmons@Benjamins-MBP-2 project2_starter-1 %

```

By carefully manipulating the ciphertext and observing the server's padding error responses, we successfully decrypted the encrypted messages without access to the key. This process demonstrated how a seemingly small vulnerability in error handling can be exploited to reveal sensitive information. Our approach systematically uncovered the plaintext, reinforcing the power of padding oracle attacks in real-world cryptographic weaknesses.

2. Bleichenbacher's Attack

Bleichenbacher's attack exploits weaknesses in RSA signature validation, particularly when using small public exponents like $e = 3$. Some implementations fail to correctly enforce strict PKCS#1 v1.5 padding, allowing attackers to build forged signatures that appear valid. This vulnerability enables unauthorized transactions by manipulating the signature verification process. In this write-up, we document our approach to constructing forged signatures for the Bank of CPSC 4200/6200, leveraging a flawed padding implementation.

The main flaw in this RSA implementation is its insecure validation of the signature format. Instead of strictly enforcing the correct padding structure, it allows signatures with missing or extra bytes to pass. This means an attacker can craft a fake signature by adjusting the message structure and taking advantage of how the system ignores unexpected data. Since RSA with $e = 3$ makes certain values easier to compute, this further simplifies forging a valid-looking signature.

We worked together in phases to ensure efficiency:

- Firstly, focusing on understanding the PKCS#1 padding structure and setting up the Python script to process and manipulate byte sequences correctly.
- Then Implemented the mathematical operations needed to construct a valid forged signature, including computing the cube root and formatting the final base64-encoded signature.

To forge the signature, we first structured the target integer by including the correct prefix (00 01), a minimal padding of one FF byte, the ASN.1 encoding, and the hash of the transfer message. The remaining 201 bytes were set to 00 to allow for a clean cube root calculation. Since RSA with $e = 3$ does not wrap around the modulus when the result is small, we could compute the cube root directly. To ensure accuracy, we used an integer cube root function from the provided roots.py library. Finally, we converted the computed integer into a base64-encoded signature and tested it against the Bank verification system to confirm its validity.

```
benjaminusimmons@Benjamins-MBP-2 project2_starter-1 % python3 bleichenbacher.py "coach+username+100.00"
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
70vEWHzTZqPp6ffcztftKt0/kdq0FN/Zyj0scnFyKIihbmew==
Ms0GJIRZEYF78SXDzxU0P04T1fj0/WNihHvznakKzN8XKGSJQLjyRuz9Q
benjaminusimmons@Benjamins-MBP-2 project2_starter-1 %
```

Through our implementation, we successfully generated forged digital signatures that passed the bank's validation system. This confirmed that the flawed padding validation could be exploited to authorize unauthorized transactions. Our approach demonstrated how an attacker could systematically manipulate signature verification to create valid signed messages.

Bank of CPSC 4200/6200 💰

Balance Transfer

Deposit an electronic transfer

Valid signature

Balance transfer initiated. 📄
(Your funds will arrive in 6-8 weeks.)

Nice work username!

By leveraging lenient padding verification and the properties of RSA with $e = 3$, we were able to forge digital signatures without knowing the private key. This exercise underscored the importance of strict padding enforcement in cryptographic protocols, as improper validation can lead to severe security vulnerabilities.